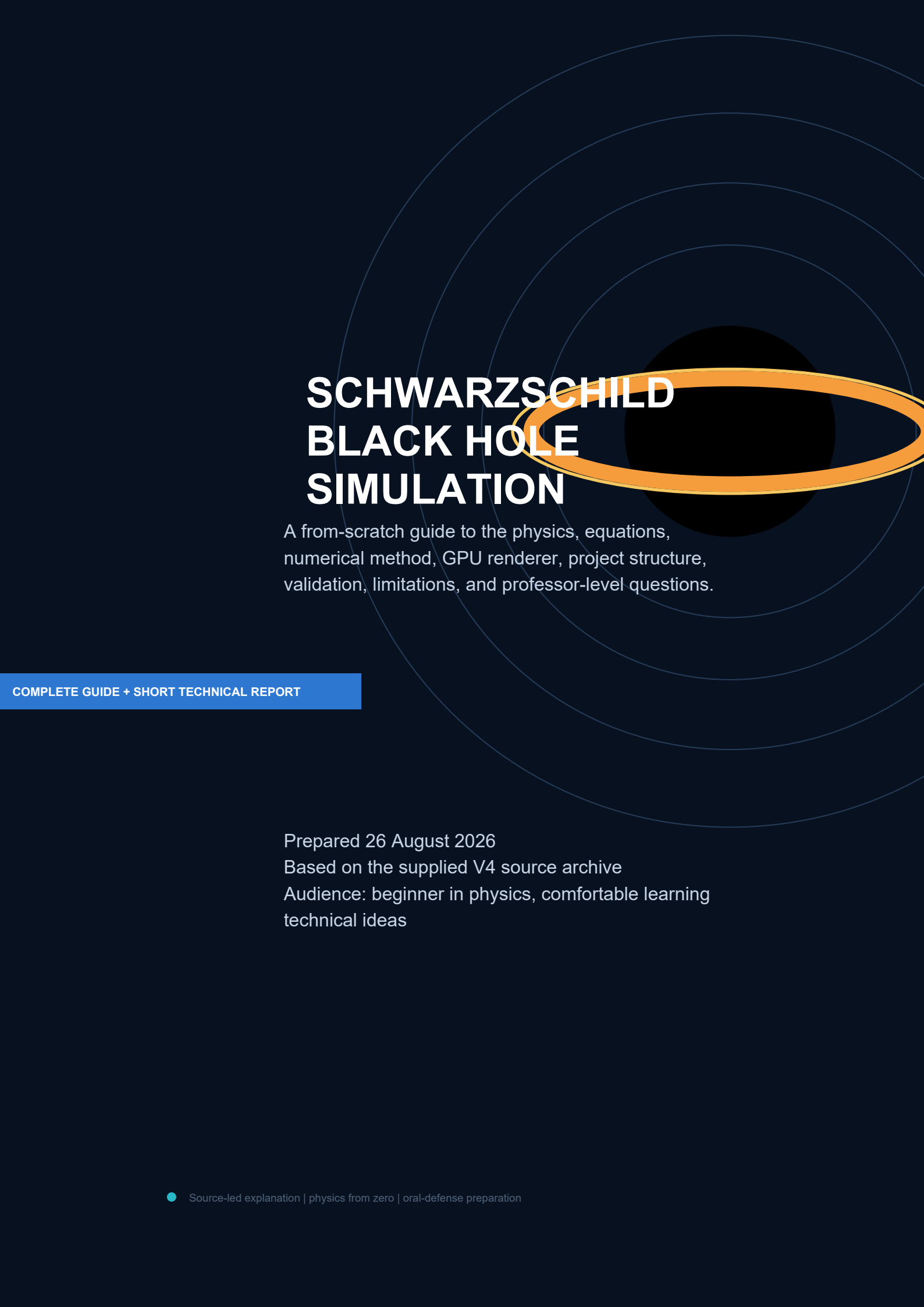




SCHWARZSCHILD BLACK HOLE SIMULATION

A from-scratch guide to the physics, equations, numerical method, GPU renderer, project structure, validation, limitations, and professor-level questions.

COMPLETE GUIDE + SHORT TECHNICAL REPORT

Prepared 26 August 2026

Based on the supplied V4 source archive

Audience: beginner in physics, comfortable learning technical ideas

Contents

0. How to use this guide	4
A 60-second explanation for your professor	4
The three most important distinctions	4
1. Physics from zero	4
1.1 What is a black hole?	4
1.2 Essential vocabulary	5
1.3 The Schwarzschild assumptions	5
1.4 Schwarzschild radius	5
1.5 Coordinate system	5
2. The Schwarzschild geometry and light	6
2.1 The metric, term by term	6
2.2 Why use dimensionless units?	6
2.3 Static-observer tetrad: turning a screen ray into a photon	6
3. What the project actually renders	7
3.1 Reverse ray tracing	7
3.2 Five possible ray states	7
3.3 Why the shadow is larger than the horizon	8
3.4 Multiple images and the bright ring	8
4. End-to-end execution flow	8
4.1 Startup	8
4.2 Per frame	8
4.3 Why a compute shader?	8
4.4 The output is not a 3D mesh of the black hole	9
5. Project structure and ownership	9
5.1 Clean separation of responsibilities	10
6. The geodesic equations used	10
6.1 General geodesic idea	10
6.2 CPU coordinate equations	10
6.3 Conserved quantities	10
6.4 Cancellation-resistant GPU radial equation	10
7. Numerical integration: RK4 plus adaptive control	11
7.1 Why numerical integration is needed	11
7.2 Classical fourth-order Runge-Kutta	11
7.3 Step doubling	11
7.4 Extra geometric step limits	11
7.5 Event interpolation and honest failures	11
8. Analytic landmarks and why they matter	12
8.1 Photon sphere	12
8.2 Critical impact parameter	12
8.3 Shadow half-angle for a finite static observer	12

8.4 Weak-field deflection	12
8.5 ISCO and disk inner edge	12
9. Accretion disk, temperature, and relativistic shading	13
9.1 Disk geometry	13
9.2 Approximate zero-torque thin-disk flux	13
9.3 Effective temperature	13
9.4 Separate physical and display profiles	13
9.5 Frequency shift g	13
9.6 Why intensity uses g^4	13
9.7 Color and procedural detail	14
10. Physical mode, Cinematic mode, and the optional grid	14
10.1 Physical does not mean complete astrophysical realism	14
10.2 Flamm's paraboloid	14
10.3 Background lensing	14
11. Default configuration translated into real scales	15
12. Validation: what was checked and what the evidence means	15
12.1 Verification versus validation	15
12.2 CPU suite rerun for this guide	15
12.3 Project-recorded GPU evidence	16
12.4 Why CPU and GPU are both needed	16
13. Limitations, assumptions, and likely criticisms	16
13.1 A subtle source-model issue	17
13.2 Good future extensions	17
14. Build, run, controls, and troubleshooting	17
14.1 Requirements	17
14.2 Standard build	17
14.3 Controls	17
14.4 Useful diagnostics	18
14.5 Common failures	18
15. Professor question bank with model answers	18
16. Short technical report	20
Title	20
Abstract	20
Objectives	20
Method	20
Emission model	20
Results	20
Conclusion	21
17. Glossary	21
18. Formula and source traceability	22
18.1 Internal references used	22

0. How to use this guide

The one-sentence answer

This is a C++17/OpenGL scientific renderer that traces light rays backward from a camera through the spacetime of a nonrotating, uncharged Schwarzschild black hole, then colors each pixel according to whether the ray falls into the hole, hits an approximate glowing disk, or escapes to a lensed background.

Read Chapters 1-4 to understand the idea without assuming physics. Chapters 5-9 connect the idea to the implementation. Chapters 10-12 give evidence, limitations, and operating instructions. Chapter 13 is an oral-defense question bank. Chapter 14 is a compact technical report you can hand to or summarize for a professor.

A 60-second explanation for your professor

The project renders the visual appearance of a Schwarzschild black hole by reverse ray tracing. For every screen pixel, an OpenGL compute shader constructs a physically valid photon direction in the camera's local orthonormal frame. It integrates the Schwarzschild null-geodesic equations using adaptive fourth-order Runge-Kutta. A ray is labeled captured, escaped, disk-hit, unresolved, or invalid. Captured rays are black; escaped rays sample a procedural sky in their bent final direction; disk-hit rays use an approximate thin-disk temperature model, relativistic frequency shift, and a bolometric g^4 intensity factor. The CPU reference and tests verify the vacuum geometry and several analytic results. The disk appearance is useful but is not a full plasma or radiative-transfer simulation.

The three most important distinctions

- **Black hole versus black-hole image:** the event horizon is not a material black sphere. The central dark region is a shadow formed by rays that cannot reach the observer.
- **Validated geometry versus approximate matter:** the Schwarzschild vacuum ray paths are the strongest scientific part. The accretion disk is a simplified emitting surface with procedural texture.
- **Physical mode versus Cinematic mode:** both bend rays the same way, but Cinematic mode adds artistic orange grading, artificial photon-ring emission, brightness boosts, and filmic tone mapping.

Naming note

The archive and repository folder spell the name as **schwarzschild**. The physicist's name and the correct scientific term are **Schwarzschild**.

1. Physics from zero

1.1 What is a black hole?

A black hole is a region of spacetime where gravity is so strong that no future-directed light signal starting inside the event horizon can reach the outside universe. General relativity does not describe gravity as a conventional pulling force. It describes mass-energy as changing spacetime geometry, and free objects and light follow the straightest possible paths in that curved geometry.

1.2 Essential vocabulary

Term	Plain-language meaning	Role here
Spacetime	The combined four-dimensional description of three space directions and time.	Photon paths are curves in spacetime, not merely bent lines in ordinary space.
Metric	A rule for measuring time, distance, and causal structure at each location.	The Schwarzschild metric defines the geometry used by the equations.
Event horizon	The one-way causal boundary of the black hole.	At $r = r_s$; numerical capture uses $r = 1.0005 r_s$, just outside the coordinate singularity.
Photon	A quantum of light. In geometric optics it is modeled as a ray.	Every pixel follows one representative photon path backward.
Null geodesic	A lightlike path through curved spacetime.	This is the differential equation integrated by CPU and GPU.
Accretion disk	Matter orbiting and falling toward a compact object, heated by dissipation.	Modeled as a simplified finite slab from 3 to $12 r_s$.
Gravitational lensing	Deflection of light caused by spacetime curvature.	It distorts the sky and creates multiple apparent disk images.
Redshift / blueshift	Observed light becomes lower / higher frequency.	Gravity and disk motion change color and brightness through g.

1.3 The Schwarzschild assumptions

The Schwarzschild solution applies to an isolated central object that is (1) nonrotating, (2) electrically uncharged, (3) spherically symmetric, and (4) surrounded by vacuum for the ray-path calculation. These assumptions make the metric static and simple enough for a clean validation target.

What Schwarzschild excludes

A real rotating black hole is described by the Kerr metric and exhibits frame dragging. This project has no spin, no Kerr geometry, no magnetic fields, no evolving spacetime, and no self-consistent plasma dynamics.

1.4 Schwarzschild radius

$$r_s = 2 G M / c^2$$

G is the gravitational constant, M is black-hole mass, and c is the speed of light. The horizon occurs at $r = r_s$ in Schwarzschild coordinates.

For the configured mass $M = 8.54 \times 10^{36}$ kg, the program derives $r_s = 12,683,881,739.7$ m = 12.684 million km = 0.0848 AU. That mass is about 4,294,759 solar masses. The same vacuum image in dimensionless r_s units applies to any Schwarzschild mass; changing M only rescales physical distances and disk thermodynamics.

1.5 Coordinate system

The project uses spherical coordinates (r, theta, phi) with the Y axis as the polar/up axis. r is distance from the center; theta is the angle down from +Y; phi rotates from +X toward +Z.

$$\begin{aligned} x &= r \sin(\theta) \cos(\phi) \\ y &= r \cos(\theta) \\ z &= r \sin(\theta) \sin(\phi) \end{aligned}$$

A coordinate chart is a labeling system, not physical scaffolding. Schwarzschild t becomes badly behaved at the horizon even though the horizon itself is not a physical singularity. The program therefore stops just outside it rather than trying to evolve these coordinates through it.

2. The Schwarzschild geometry and light

2.1 The metric, term by term

$$ds^2 = -f dt^2 + f^{-1} dr^2 + r^2(d\theta^2 + \sin^2(\theta) d\phi^2)$$

$$f(r) = 1 - r_s/r$$

The project uses geometrized, dimensionless coordinates for ray motion: $r_s = 1$ and $\lambda_{\text{dimensionless}} = \lambda/r_s$. In those units $f = 1 - 1/r$. The first term controls time, the second radial distance, and the final two angular motion. The minus sign on the time term expresses the Lorentzian distinction between time and space.

For light, $ds^2 = 0$. This is the **null condition**. It does not mean the photon travels no distance in ordinary space; it means the spacetime interval along an ideal light ray is zero. The code evaluates this as a diagnostic invariant.

$$g(k,k) = -f (k^t)^2 + (k^r)^2/f + r^2[(k^\theta)^2 + \sin^2(\theta)(k^\phi)^2] = 0$$

$k^\mu = dx^\mu/d\lambda$ is the photon four-wavevector (or tangent vector). Dots/primes in this guide mean derivatives with respect to the affine parameter λ .

2.2 Why use dimensionless units?

Setting $r_s = 1$ removes very large SI distances from the geodesic arithmetic, improves numerical conditioning, and exposes a key physical symmetry: every uncharged, nonrotating black hole has the same vacuum lensing pattern when all distances are measured in units of its own r_s . The C++ camera position is stored in metres but divided by r_s before upload to the GPU.

2.3 Static-observer tetrad: turning a screen ray into a photon

A pixel direction is initially an ordinary unit vector in the camera's local laboratory. It cannot be inserted directly as coordinate derivatives because spherical coordinates and gravitational time dilation change the basis lengths. A **tetrad** is a local orthonormal frame: locally, the camera measures directions as if it were in flat space. The code resolves the pixel direction into (n_r, n_θ, n_ϕ) and transforms it to coordinate components.

$$k^t = E_{\text{local}} / \sqrt{f}$$

$$k^r = E_{\text{local}} \sqrt{f} n_r$$

$$k^\theta = E_{\text{local}} n_\theta / r$$

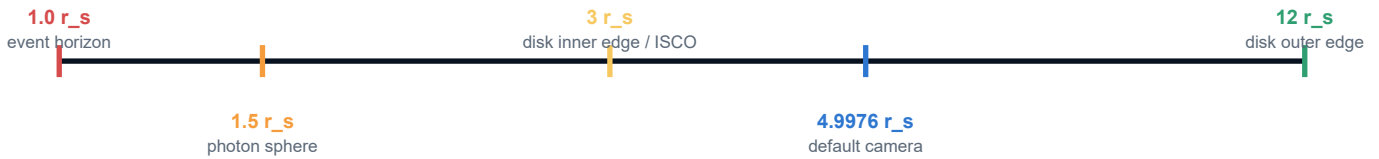
$$k^\phi = E_{\text{local}} n_\phi / [r \sin(\theta)]$$

$$E_{\text{inf}} = f k^t$$

E_{local} can be chosen as 1 because multiplying every component by a positive constant only re-parameterizes the same null path. The transformation guarantees a normalized local direction and makes the initial null norm approximately zero to floating-point accuracy.

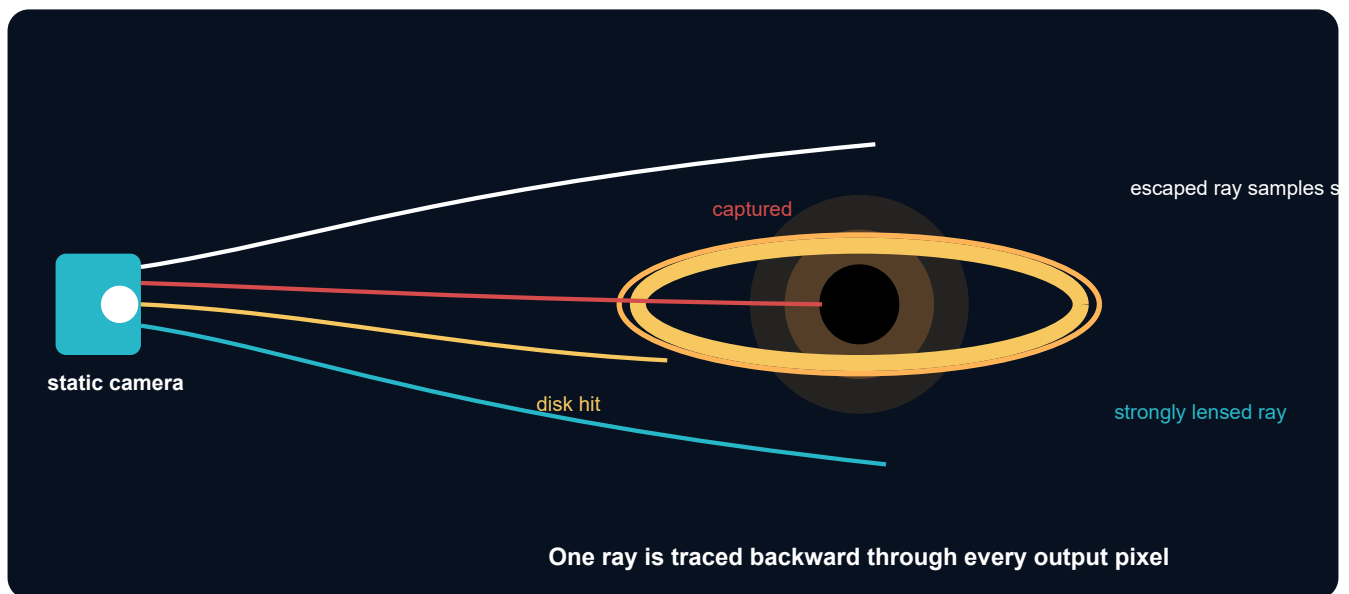
Why the camera cannot cross the horizon

A static observer would need infinite acceleration at $r = r_s$ and cannot exist at or inside it. The controls clamp the camera to $r \geq 1.05 r_s$. This is a physical restriction of a static camera, not an arbitrary visual limitation.



Logarithmic spacing for legibility; distances are measured from the center.

3. What the project actually renders



3.1 Reverse ray tracing

The physically correct question for an image is: 'From which source did the light reaching this pixel come?' Forward-tracing photons from every source would waste almost all work because very few rays hit the camera. The program therefore starts at the camera and integrates each ray backward. General-relativistic light paths are reversible in this static spacetime, so this efficiently finds the apparent source.

3.2 Five possible ray states

Status	Meaning	Pixel treatment
Captured	The ray crosses the capture surface at $1.0005 r_s$.	Black: it belongs to the shadow.
Escaped	The ray reaches a large radius while moving outward.	Sample the procedural sky using the bent final direction, or use a uniform dark validation background.
Disk hit	The path segment enters/crosses the finite inclined disk slab within $3-12 r_s$.	Evaluate disk temperature, frequency shift, texture, and intensity.
Unresolved	The integrator cannot satisfy tolerance or exhausts its step budget.	Yellow in validation overlay; dark blue otherwise. Never counted as captured.
Invalid	Coordinates or values become nonfinite or invalid.	Magenta in validation overlay; dark purple otherwise. Never counted as captured.

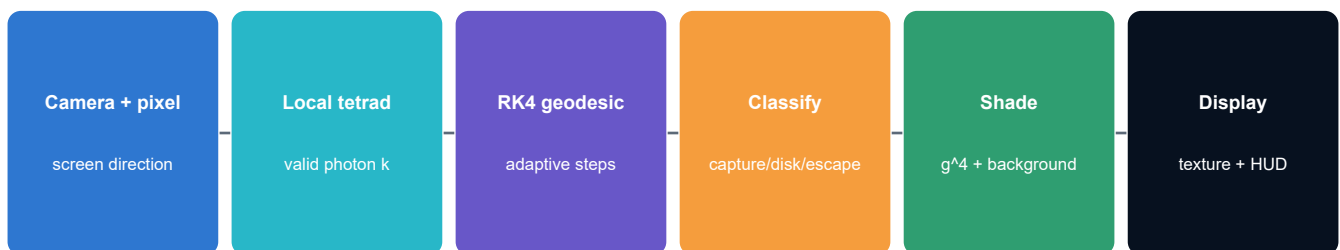
3.3 Why the shadow is larger than the horizon

The edge of the observed shadow is controlled by near-critical photon trajectories, not by the physical radius of the event horizon projected like a solid ball. Rays with sufficiently small impact parameter are captured. The unstable photon sphere at $1.5 r_s$ separates qualitatively different paths and produces a critical impact parameter $b_{\text{crit}} = 2.598... r_s$ for rays arriving from far away.

3.4 Multiple images and the bright ring

Strongly bent rays can wrap partly around the hole before reaching the camera. That makes the far side of the disk appear above and below the shadow and can create higher-order images. In Physical mode, any narrow bright structure must come from lensed source emission and sampling. In Cinematic mode, the shader additionally paints an artificial photon-ring glow near $r = 1.5 r_s$; it is explicitly an artistic enhancement.

4. End-to-end execution flow



GPU path: `geodesic.comp` executes the middle four stages for many pixels in parallel.

4.1 Startup

- `main()` creates the validated default configuration and parses command-line modes such as captures, GPU export, thickness validation, and temperature validation.
- Camera constructs its default orbit position in SI metres, centered on the origin.
- Engine creates a GLFW OpenGL 4.3 window, loads GLEW, compiles shaders, creates the output texture, uniform buffers, diagnostic buffers, fullscreen quad, optional grid, and ImGui HUD.

4.2 Per frame

- Choose compute resolution. Normal interaction uses half framebuffer dimensions and halves again while moving; full-resolution still mode uses every framebuffer pixel.
- Upload camera, disk, and integration configuration through uniform-buffer objects (UBOs). Camera position is divided by r_s ; disk radii and thickness are already in r_s units.
- Clear diagnostic counters and dispatch `geodesic.comp` in 16×16 GPU work groups.
- Each invocation handles one pixel: build a local screen ray, integrate it, classify it, and write RGBA color plus optional diagnostics.
- Issue an OpenGL memory barrier, read status counters for the HUD, draw the computed texture on a fullscreen quad, optionally overlay Flamm's paraboloid, draw the HUD, and swap buffers.

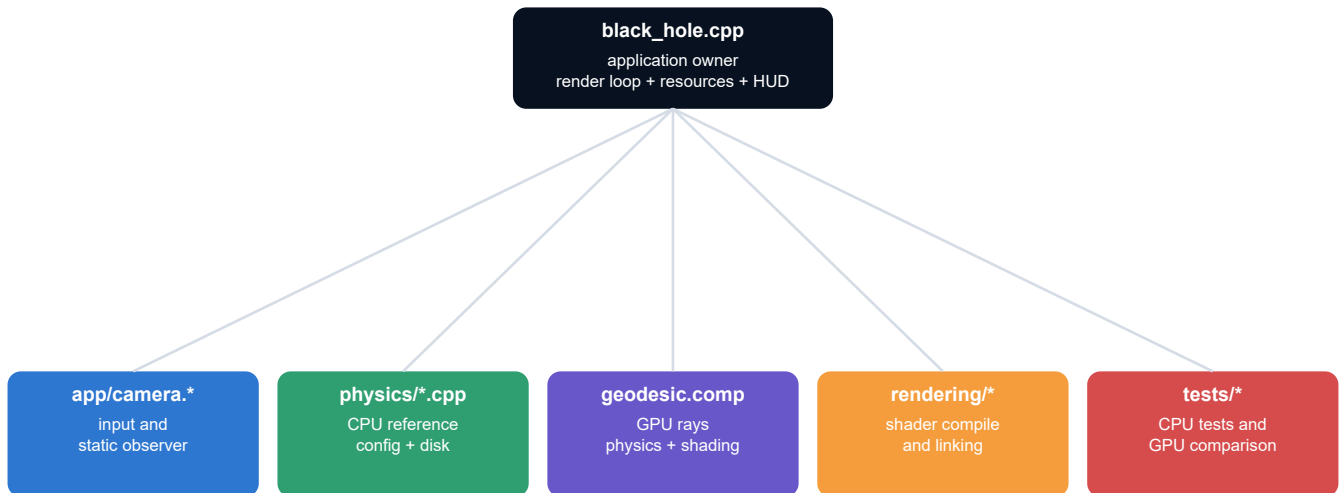
4.3 Why a compute shader?

Every pixel's ray is independent until the final counters are accumulated, which is ideal data-parallel work. A compute shader lets thousands of GPU threads solve rays simultaneously. The CPU remains responsible for window management, configuration, input, resource ownership, and reference validation.

4.4 The output is not a 3D mesh of the black hole

The main black-hole image is a generated 2D texture. It looks three-dimensional because the ray equations and shading encode the geometry. Only the optional Flamm overlay is conventional line geometry. This distinction is a common professor question.

5. Project structure and ownership



CMake builds reusable CPU physics, two renderers, and two validation executables.

Path	Purpose	Important details
CMakeLists.txt	Build graph	C++17; reusable physics library; 3D and historical 2D apps; CPU validator; GPU comparator; warning flags; shader copying.
vcpkg.json	Dependencies	GLEW, GLFW3, GLM, ImGui.
black_hole.cpp	Main 3D application	OpenGL resources, compute dispatch, configuration upload, HUD, runtime modes, exports, captures, diagnostics, cleanup.
geodesic.comp	Production GPU kernel	Per-pixel ray initialization, adaptive RK4 geodesics, disk intersection, background, shading, tone mapping, status output.
physics/schwarzschild.*	CPU reference	Coordinates, r_s , metric, tetrad initialization, null norm, invariants, exact coordinate RHS, RK4, adaptive vacuum tracing.
physics/accretion_disk.*	Disk reference	Thin-disk flux, effective temperature, circular-emitter frequency shift, g^4 factor.
physics/simulation_config.*	Central configuration	Defaults and validity constraints for mass, camera, disk, integrator, resolution, modes.
app/camera.*	Interaction	Orbit, zoom, reset, mode toggles, disk inclination/thickness/rotation controls, static-radius clamp.
rendering/shader_loader.*	Shader utilities	Read files, compile shaders, link programs, report errors, release objects on failure.
tests/schwarzschild_validation.cpp	Scientific CPU tests	Analytic and convergence tests without opening a window.
tests/gpu_validation_compare.cpp	Cross-check tool	Compares exported GPU classifications/invariants/temperatures against CPU or analytic expectations.
2D_lensing.cpp	Historical 2D viewer	Separate, non-validation visualization; retained for context, not the authoritative 3D path.
grid.vert / grid.frag	Grid overlay shaders	Render Flamm's paraboloid lines.
README.md	User documentation	Scope, build, run, controls, architecture, limitations.
PHYSICS_VALIDATION.md	Physics contract	Equations, conventions, tolerances, evidence, and limitations.
VALIDATION_REPORT.md	Recorded audit	Fresh-build commands, hardware, CPU/GPU results, interactions, captures, timing.

5.1 Clean separation of responsibilities

The design intentionally keeps dependency-free reference mathematics in physics/, real-time float implementation in the compute shader, and presentation/resource management in the app. This makes it possible to test mathematical behavior without an OpenGL window and to compare GPU output against an independent double-precision implementation.

6. The geodesic equations used

6.1 General geodesic idea

A geodesic satisfies $d^2 x^\mu / d\lambda^2 + \Gamma^\mu_{\alpha\beta} (dx^\alpha / d\lambda)(dx^\beta / d\lambda) = 0$. The Γ terms, called Christoffel symbols, encode how the coordinate basis changes in curved spacetime. The project does not numerically compute a general Christoffel tensor every step; it uses the Schwarzschild equations simplified into explicit component formulas.

6.2 CPU coordinate equations

$$\begin{aligned} t'' &= -t' r' / (r^2 f) \\ r'' &= -f(t')^2 / (2r^2) + (r')^2 / (2r^2 f) + r f[(\theta')^2 + \sin^2(\theta)(\phi')^2] \\ \theta'' &= -2 r' \theta' / r + \sin(\theta)\cos(\theta)(\phi')^2 \\ \phi'' &= -2 r' \phi' / r - 2 \cot(\theta) \theta' \phi' \end{aligned}$$

The CPU implementation keeps this direct coordinate form as the readable reference. It evolves eight first-order quantities: position (t, r, θ, ϕ) and their four derivatives.

6.3 Conserved quantities

$$\begin{aligned} E &= f t' \\ L^2 &= r^4[(\theta')^2 + \sin^2(\theta)(\phi')^2] \\ L_z &= r^2 \sin^2(\theta) \phi' \\ b &= L/E \end{aligned}$$

E is the photon energy associated with time-translation symmetry; L^2 is total angular momentum squared; L_z is its component about the coordinate Y axis; b is impact parameter. In a spherically symmetric spacetime, total angular momentum and the orbital plane remain constant. Tests watch their numerical drift.

6.4 Cancellation-resistant GPU radial equation

$$r'' = L^2 (r - 1.5) / r^4$$

For a null ray, substituting the null constraint and conserved L^2 into the direct radial equation gives this algebraically equivalent form. Near the horizon, the first two large direct terms can nearly cancel in 32-bit floating point and lose significant digits. The reduced expression is therefore more stable on the GPU. It also makes the photon sphere visible: $r'' = 0$ for a purely tangential critical ray at $r = 1.5$.

Why total L^2 matters

Using only L_z would work only for rays confined to one equatorial plane. A 3D camera generates off-equatorial rays, so the shader correctly uses total angular momentum squared.

7. Numerical integration: RK4 plus adaptive control

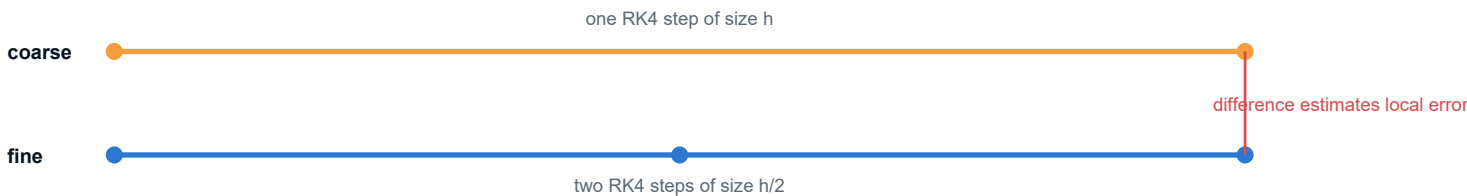
7.1 Why numerical integration is needed

Most camera rays in this 3D setup do not have a convenient closed-form position as a function of the integration parameter. The program therefore takes many small numerical steps. Strong curvature near the photon sphere and horizon needs smaller steps; weak fields far away allow larger steps.

7.2 Classical fourth-order Runge-Kutta

```
k1 = F(y_n)
k2 = F(y_n + h k1/2)
k3 = F(y_n + h k2/2)
k4 = F(y_n + h k3)
y_(n+1) = y_n + h(k1 + 2k2 + 2k3 + k4)/6
```

Here y is the photon state, $F(y)$ is the geodesic right-hand side, and h is the affine-parameter step. RK4 has global error proportional to h^4 for a smooth problem, so halving h should reduce the error by about $2^4 = 16$. The project tests and observes a ratio of 15.9824.



7.3 Step doubling

The integrator computes one RK4 step of size h and, independently, two RK4 half-steps. The difference estimates local error. The accepted result is the more accurate two-half-step state. For RK4 step doubling, the difference is approximately 15 times the fine solution's error, so the code divides the normalized difference by 15.

```
scale_i = abs_tol + rel_tol max(|y_coarse,i|, |y_fine,i|)
error = max_i [ |y_coarse,i - y_fine,i| / scale_i ] / 15
```

If $\text{error} \leq 1$, the step is accepted. Otherwise h shrinks and the step is retried, up to ten attempts. After acceptance, h is grown or reduced by a safety-controlled factor approximately $0.9 \text{error}^{(-1/5)}$, clamped to avoid violent changes. The exponent $1/5$ follows the fifth-order local error behavior of RK4.

7.4 Extra geometric step limits

The GPU also limits h using a curvature scale $0.02 r^2$ and one quarter of the remaining horizon distance divided by an inward-speed bound. These are safety ceilings; step doubling remains the actual error controller. Defaults are $\text{abs_tol} = \text{rel_tol} = 2 \times 10^{-5}$, $h_{\text{min}} = 10^{-5}$, $h_{\text{max}} = 0.10$, and at most 24,000 integration iterations.

7.5 Event interpolation and honest failures

A finite step can cross a thin target between endpoints. The disk routine interpolates entry into the slab or crossing of its midplane and then checks radial bounds. Horizon crossings are linearly interpolated between previous and current ray states. If accuracy cannot be met or values become invalid, the ray remains unresolved/invalid instead of being silently colored black. That prevents numerical errors from enlarging the shadow.

8. Analytic landmarks and why they matter

8.1 Photon sphere

$$r_{\text{ph}} = 3GM/c^2 = 1.5 r_s$$

At this radius, an ideal photon can orbit circularly. The orbit is unstable: a tiny perturbation makes it fall inward or escape outward. Near-critical rays can spend a long time near this radius, so they are numerically demanding and visually important.

8.2 Critical impact parameter

$$b_{\text{crit}} = 3 \sqrt{3} GM/c^2 = (3 \sqrt{3}/2) r_s = 2.598076211... r_s$$

For a ray sent from far away toward the hole, b below this value leads to capture and b above it leads to escape. The project validates both the exact value and classifications at $b = 2.4$ and $b = 3.0$.

8.3 Shadow half-angle for a finite static observer

$$\alpha = \arcsin[b_{\text{crit}} \sqrt{f(r_o)} / r_o]$$

At the default camera radius $r_o = 4.9976 r_s$, this gives $\alpha = 27.7071984$ degrees. The full angular diameter of the shadow is about 55.414 degrees. The formula includes the local static-observer frame; simply using b_{crit}/r_o would miss the gravitational factor $\sqrt{f}$.

8.4 Weak-field deflection

$$\delta \text{ approximately } 4GM/(b c^2) = 2 r_s/b$$

For $b = 100 r_s$, the leading estimate is 0.02 rad. The CPU ray gives 0.020299965 rad, a 1.50% difference, which is expected because the formula shown is only the first weak-field term. The project tolerance is 2.5%.

8.5 ISCO and disk inner edge

$$r_{\text{ISCO}} = 6GM/c^2 = 3 r_s$$

ISCO means innermost stable circular orbit. In Schwarzschild spacetime, circular timelike orbits inside $3 r_s$ are unstable. The disk model therefore sets its inner edge to $3 r_s$. This is about matter orbits, not the photon sphere, which is at $1.5 r_s$.

Landmark	Radius	Meaning
Event horizon	$1.0 r_s$	One-way causal boundary.
Photon sphere	$1.5 r_s$	Unstable circular null orbit.
Critical impact parameter	$2.598 r_s$	Capture/escape boundary as seen from infinity; not a physical radius.
ISCO	$3.0 r_s$	Innermost stable circular matter orbit and chosen disk inner edge.
Default camera	$4.9976 r_s$	Static observer location.
Disk outer edge	$12.0 r_s$	Chosen visualization/model cutoff.

9. Accretion disk, temperature, and relativistic shading

9.1 Disk geometry

The disk is a finite inclined slab, not an infinitesimally thin plane and not simulated fluid. Its inner radius is $3 r_s$, outer radius $12 r_s$, default inclination 20 degrees, and default half-thickness $0.12 r_s$. Its normal is obtained by tilting the original Y-up disk around the Z axis. A segment hits when it enters the slab or crosses the midplane and the interpolated in-plane radius is within the annulus.

9.2 Approximate zero-torque thin-disk flux

$$F(r) = [3 G M \dot{M} / (8 \pi r^3)] [1 - \sqrt{r_{in}/r}]$$

F is emitted power per unit surface area, $\dot{M}$ is accretion rate, and the boundary factor forces F to zero at the inner edge, representing an idealized zero-torque condition. The r^{-3} behavior makes outer regions cooler. This is a Newtonian-style thin-disk approximation placed on top of exact Schwarzschild vacuum ray paths.

9.3 Effective temperature

$$F = \sigma T_{eff}^4 \Rightarrow T_{eff} = (F/\sigma)^{1/4}$$

σ is the Stefan-Boltzmann constant. C++ packs the dimensional part into a temperature scale and the shader evaluates the dimensionless radial shape:

$$\begin{aligned} T_{scale} &= [3 G M \dot{M} / (8 \pi \sigma r_s^3)]^{1/4} \\ shape(r_{bar}) &= [1 - \sqrt{r_{in_bar}/r_{bar}}] / r_{bar}^3 \\ T_{local} &= T_{scale} shape^{1/4} \end{aligned}$$

Bars mean radii measured in r_s units. Because temperature depends on the fourth root, increasing $\dot{M}$ by 10 multiplies T by only $10^{1/4} = 1.778279$. The tests verify this. At $4.5 r_s$, the default temperature is 5866.05 K.

9.4 Separate physical and display profiles

The shader uses the physical r^{-3} shape only for Kelvin temperature. A separate normalized profile $(r/r_{in})^{-3}$ controls exposure to preserve a useful image. Mixing these would add an erroneous factor of r_{in}^3 to flux and $r_{in}^{3/4}$ to temperature. The code deliberately keeps them separate.

9.5 Frequency shift g

$$g = \nu_{observer} / \nu_{emitter} = [-k \cdot u_{observer}] / [-k \cdot u_{emitter}]$$

u is an observer or emitter four-velocity, k is the photon four-vector, and the dot is the spacetime metric inner product. For a static observer and a circular Schwarzschild emitter, the implementation uses:

$$\begin{aligned} u_o^t &= 1/\sqrt{1 - 1/r_o} \\ u_e^t &= 1/\sqrt{1 - 1.5/r_e} \\ \Omega &= \sqrt{1/(2 r_e^3)} \\ g &= (E u_o^t) / \{u_e^t [E - s \Omega L_{axis}]\} \end{aligned}$$

$s = +1$ or -1 chooses disk rotation direction, and L_{axis} is photon angular momentum projected onto the disk normal. This single ratio includes gravitational and motion-related Doppler shifts for the assumed observers. At the test point, reversing relative motion swaps approximately $g = 1.103686$ and $g = 0.778389$.

9.6 Why intensity uses g^4

$$I_{obs,bolometric} = g^4 I_{emit,bolometric}$$

Liouville's theorem gives invariance of I_{ν}/ν^3 along a vacuum ray. Integrating over frequency contributes one more factor of g , producing g^4 for bolometric intensity. This makes the approaching side brighter and the receding side dimmer. It is not a full camera-band spectrum or radiative-transfer solution.

9.7 Color and procedural detail

Physical mode maps shifted effective temperature through a fast blackbody-to-RGB approximation, then multiplies by edge fade, display exposure, procedural surface density, and g^4 . Noise, ridges, rings, clumps, and differential rotation make the disk visually structured. They modulate emissivity only; they never change ray motion.

10. Physical mode, Cinematic mode, and the optional grid

Feature	Physical mode	Cinematic mode
Null geodesics	Validated GPU integration	Same integration
Capture / lensed sky	Yes	Yes
Disk local temperature	Approximate thin-disk model	Same base
Frequency shift and g^4	Yes	Yes
Color	Approximate blackbody RGB	Fixed orange/gold grading
Artificial photon-ring emission	No	Yes, Gaussian proximity to $1.5 r_s$ plus winding boost
Higher-order brightness boost	No	Yes
Tone mapping	Simple color/(1+color)	Hue-preserving ACES-style filmic curve

10.1 Physical does not mean complete astrophysical realism

Physical mode means the known artistic additions are disabled and the documented approximations are used consistently. It still lacks frequency-dependent radiation, absorption, scattering, optical-depth integration, polarization, magnetohydrodynamics, and realistic plasma emissivity. A defensible description is 'validated Schwarzschild null-geodesic renderer with an approximate disk emitter,' not 'fully accurate black-hole simulation.'

10.2 Flamm's paraboloid

$$z(r) = 2 \sqrt{r_s(r - r_s)}$$

This surface embeds the spatial geometry of a constant-time equatorial Schwarzschild slice into ordinary Euclidean 3D space. It is a teaching visualization of one two-dimensional spatial slice. It is not the event horizon, not a gravity well that light physically rolls down, and not a literal picture of four-dimensional spacetime curvature.

10.3 Background lensing

The procedural stars and galactic band are evaluated using the ray's final direction after it escapes. That is why the sky pattern bends around the hole. Validation can replace the sky with uniform near-black so no background detail is mistaken for black-hole emission.

11. Default configuration translated into real scales

Parameter	Code default	Physical interpretation
Black-hole mass	8.54×10^{36} kg	About 4,294,759 solar masses.
Schwarzschild radius	1 r_s	12.684 million km = 0.0848 AU; light crossing time 42.31 s.
Camera radius	4.9976 r_s	63.389 million km = 0.424 AU from center.
Camera minimum	1.05 r_s	Static-observer safety margin outside the horizon.
Vertical FOV	60 degrees	Total vertical angular field seen by the camera.
Disk inner edge	3 r_s	38.052 million km = 0.254 AU; Schwarzschild ISCO.
Disk outer edge	12 r_s	152.207 million km = 1.017 AU; a chosen cutoff.
Disk half-thickness	0.12 r_s	1.522 million km each side; total thickness 0.24 r_s .
Disk inclination	20 degrees	Tilt around Z relative to the original X-Z plane.
Accretion rate	1.0×10^{15} kg/s	Input to the approximate temperature model.
Capture radius	1.0005 r_s	Numerical event surface just outside horizon.
Escape radius	$\geq 80 r_s$	Also enlarged based on camera and disk: $\max(80, 2.5 \text{ camera radius}, 8 \text{ disk outer radius})$.
GPU tolerance	$\text{abs} = \text{rel} = 2 \times 10^{-5}$	Practical 32-bit strong-field error target.
Step range	10^{-5} to 0.10	Dimensionless affine-parameter steps.
Step budget	24,000	Maximum loop iterations per ray.
Nominal resolution	1280 x 720	Interactive compute texture is normally lower; full-resolution still is optional.

Scale invariance

The vacuum ray equations do not contain the default mass after normalization. A stellar-mass and supermassive Schwarzschild black hole have the same dimensionless shadow geometry at the same r/r_s . Their metre scales, time scales, and disk thermodynamics differ.

12. Validation: what was checked and what the evidence means

12.1 Verification versus validation

Verification asks whether the code solves the intended equations correctly. Validation asks whether those equations are an adequate model of reality for the claimed purpose. This project has strong verification of Schwarzschild vacuum rays. Its disk model is openly limited, so astrophysical validation of a real plasma is outside scope.

12.2 CPU suite rerun for this guide

On 26 August 2026, the dependency-free target was configured with MSVC 19.50 using CMake/Ninja, built, and executed from the supplied archive. CTest reported 1/1 passed, and the standalone report ended Overall: PASS.

Check	Measured result	Interpretation
Initial null condition	$\max g(k,k) = 6.66 \times 10^{-16}$ over 665 rays	Tetrad initialization produces lightlike directions to double precision.
Energy conservation	max drift 3.77×10^{-15}	Time symmetry is respected numerically.
Total L^2 conservation	max drift 5.61×10^{-13}	Spherical symmetry and 3D angular motion are stable.
Orbital plane	max vector drift 4.59×10^{-14}	Rays remain in their invariant plane.
Photon sphere	$\max r - 1.5 = 0$ for test	Critical circular null orbit is reproduced.
b_{crit}	2.59807621135 r_s	Matches analytic $3 \sqrt{3}/2$.

Check	Measured result	Interpretation
Capture/escape	$b=2.4$ captured; $b=3.0$ escaped	Classification straddles the analytic boundary correctly.
Weak deflection	0.020299965 rad at $b=100$	Within 1.50% of leading 0.02 rad.
Shadow angle	27.7071984 degrees	Matches the analytic finite-observer result.
RK4 order	error reduction 15.9824	Consistent with fourth-order convergence.
Disk temperature	$T(4.5\ r_s) = 5866.05094\ \text{K}$	CPU implementation follows stated flux and Stefan-Boltzmann equations.
Mdot scaling	$T(10\ \text{Mdot})/T = 1.77827941$	Matches $10^{1/4}$.

12.3 Project-recorded GPU evidence

The repository's `VALIDATION_REPORT.md` records a clean Windows/OpenGL audit on 23 August 2026 using an NVIDIA RTX 3050 Ti Laptop GPU. Those GPU/windowed checks were **not rerun while producing this guide**; the following values are traceable to the supplied report.

GPU check	Recorded outcome
64 x 48 CPU/GPU classification	3,072 rays; 2 boundary differences, allowance 8; 0 unresolved; 0 invalid.
GPU invariants	max initial null residual 6.01×10^{-7} ; energy span 0; max escaped final null residual 2.97641×10^{-6} .
Temperature samples	Five radii matched CPU with maximum relative error 7.06×10^{-8} .
Thickness regression	Changing half-thickness 0.02 $\rightarrow$ 0.75 r_s changed 4,952/19,200 classifications and disk hits 5,726 $\rightarrow$ 10,678.
Build and interaction	Debug/Release builds, smoke tests, resize checks, captures, and OpenGL error count recorded as PASS/zero.

12.4 Why CPU and GPU are both needed

The CPU reference uses double precision and readable equations; the production shader uses float and a cancellation-resistant equivalent radial formula. Agreement with analytic results and an independently coded reference is much more convincing than testing the shader against itself.

13. Limitations, assumptions, and likely criticisms

Limitation	Why it matters	How to state it honestly
No Kerr spin	Most astrophysical black holes rotate; spin changes shadow asymmetry, photon regions, disk ISCO, and frame dragging.	This is a Schwarzschild baseline, not a rotating-black-hole renderer.
No radiative transfer	Real images depend on emission, absorption, scattering, optical depth, and camera frequency.	g^4 is a bolometric vacuum propagation factor, not a complete camera-band solution.
Approximate disk	The finite slab, procedural density, and zero-torque thin-disk temperature are not self-consistent plasma dynamics.	The disk is an illustrative emitter layered on validated ray geometry.
RGB approximation	Temperature-to-RGB is not a sampled blackbody spectrum through detector response.	Color is informative but not spectrally predictive.
32-bit GPU float	Near-critical rays are sensitive and coordinate terms become difficult near the horizon.	Double CPU tests are the reference; failures remain explicit.
Finite capture surface	The code stops at $1.0005\ r_s$, not exactly at 1.0.	This avoids the Schwarzschild-coordinate singularity and approximates horizon capture.
Finite resolution	Rasterized shadow edges can differ by a few pixels and narrow rings may alias.	The deterministic comparison allows a small number of boundary mismatches.
Procedural background	It is not a calibrated real sky map.	It demonstrates lensing direction, not astronomical sky reconstruction.
Static camera	No free-falling observer, aberration, or camera time evolution.	The local tetrad is for a static observer outside the horizon.
No license selected	Redistribution rights are unclear.	Treat the source as no-license/all-rights-reserved until the owner chooses a license.

13.1 A subtle source-model issue

The default mass is comparable to the Galactic-center black hole, but the documentation correctly warns that a quiescent source such as Sagittarius A* is usually modeled as a hot, optically thin, radiatively inefficient plasma. A cool optically thick thin disk is therefore not a faithful source-specific model. The mass scale does not make the disk astrophysically equivalent to that object.

13.2 Good future extensions

- Kerr metric with spin, frame dragging, spin-dependent ISCO, and a carefully validated camera tetrad.
- Frequency-dependent radiative transfer through an optically thin plasma, including absorption and emissivity along each ray.
- Polarization transport and magnetic-field-informed synchrotron emission.
- Higher precision or mixed precision near critical trajectories, plus event-aware or Hamiltonian integrators.
- Real sky textures, detector response, and reproducible image-comparison datasets.
- Automated GPU validation in CI on representative vendors, while preserving the dependency-free CPU tests.

14. Build, run, controls, and troubleshooting

14.1 Requirements

- Windows 10/11; GPU driver supporting OpenGL 4.3 or newer.
- Visual Studio C++ desktop workload, CMake 3.21+, Git, and vcpkg.
- Manifest dependencies: GLEW, GLFW3, GLM, and ImGui.

14.2 Standard build

```
cmake -S . -B build -A x64 -DCMAKE_TOOLCHAIN_FILE=C:/vcpkg/scripts/buildsystems/vcpkg.cmake
-DVCPKG_TARGET_TRIPLET=x64-windows
cmake --build build --config Release
ctest --test-dir build -C Release --output-on-failure
```

Run BlackHole3D.exe from the configuration output directory because CMake copies shader files next to the executable.

14.3 Controls

Input	Action
Left or middle drag	Orbit centered camera.
Mouse wheel	Zoom between 1.05 and 80 r_s .
M	Toggle Physical/Cinematic.
D	Show/hide disk.
B	Procedural sky/uniform dark background.
G	Toggle Flamm's paraboloid overlay.
V	Toggle validation-failure colors.
R	Reverse disk rotation.
I / Shift+I	Increase/decrease inclination.
T / Shift+T	Increase/decrease half-thickness.
Home	Reset known view.

14.4 Useful diagnostics

Command	Purpose
SchwarzschildValidation.exe	Dependency-free CPU scientific suite.
BlackHole3D.exe --validation-export	Generate deterministic per-pixel GPU classification/invariant CSV.
GpuValidationCompare.exe gpu_validation.csv	Compare GPU export to CPU reference and analytic shadow.
--thickness-validation-export	Generate thin/thick status maps for slab regression.
--temperature-validation-export	Probe production-shader temperature and Doppler reversal.
--interaction-smoke	Check controls, motion stability, and resizing.
--performance-smoke	Measure compute time with fixed protocol.
--capture-physical / --capture-cinematic / --capture-vacuum / --capture-grid	Produce reproducible BMP captures.

14.5 Common failures

- Window creation failure: confirm OpenGL 4.3 support and update the driver.
- Shader file not found: run from build/Release or build/Debug where CMake copied *.comp, *.vert, and *.frag.
- Wrong laptop GPU: select the executable as High performance in Windows Graphics settings and confirm the startup renderer line.
- Yellow/magenta pixels: these are deliberate unresolved/invalid diagnostics. Export validation data before changing tolerances.

15. Professor question bank with model answers

Use these answers as concepts, not as a script. A strong defense states both what the program achieves and what it does not claim.

Q1. What is the project about?

It visualizes how light propagates around an ideal Schwarzschild black hole. It is a ray renderer and numerical physics demonstration, not an evolving matter simulation.

Q2. Why is the center black?

Those pixels correspond to backward rays that cross the capture surface and therefore cannot connect the camera to a luminous source.

Q3. Is the black circle the event horizon?

Not directly. It is the observed shadow, controlled by captured trajectories and the photon region; its apparent angular size is larger than the projected horizon.

Q4. Why trace rays backward?

It guarantees that every computed ray contributes to a pixel. Forward emission would waste almost all photons.

Q5. What is Schwarzschild spacetime?

The exact exterior vacuum solution for a static, spherical, uncharged mass. Its metric supplies the ray equations.

Q6. What makes a ray a photon ray?

Its tangent obeys the null constraint $g(k,k)=0$. Tetrad initialization enforces this for the local screen direction.

Q7. What is an affine parameter?

A monotonically increasing parameter along a geodesic that makes the geodesic equation take its standard form. Its absolute scaling does not change the path.

Q8. Why set $r_s = 1$?

It makes the equations dimensionless, avoids extreme SI scales, and exposes mass-scale invariance of Schwarzschild vacuum lensing.

Q9. What numerical method is used?

Classical RK4 with step doubling for local error estimation and adaptive step size.

Q10. Why not fixed steps?

Curvature and trajectory sensitivity vary greatly. Fixed small steps waste work far away; fixed large steps miss strong-field behavior.

Q11. What proves RK4 is implemented correctly?

A step-halving convergence test shows an error reduction of 15.9824, close to the fourth-order expectation of 16.

Q12. Why is the GPU radial equation different from the CPU one?

It is algebraically equivalent for null rays after using the constraint and conserved L^2 , but avoids catastrophic cancellation in float near the horizon.

Q13. What is the photon sphere?

The unstable circular light orbit at $1.5 r_s$. Near it, rays can wind around the hole before escape or capture.

Q14. What is the critical impact parameter?

$b_{\text{crit}} = 2.598076 r_s$ separates capture and escape for incoming rays from infinity.

Q15. What is ISCO?

The innermost stable circular orbit for matter. For Schwarzschild it is $3 r_s$, used as the disk inner edge.

Q16. How is disk temperature computed?

A zero-torque thin-disk flux F proportional to $M \dot{M} r^{-3} [1 - \sqrt{r_{\text{in}}/r}]$ is converted by $T = (F/\sigma)^{1/4}$.

Q17. What is g ?

The observed-to-emitted frequency ratio computed from photon and observer/emitter four-vectors. $g > 1$ is blueshift; $g < 1$ is redshift.

Q18. Why g^4 ?

I_{ν}/ν^3 is invariant along vacuum rays; integrating over frequency produces a fourth power for bolometric intensity.

Q19. Does disk rotation change local temperature?

No in this model. Rotation changes observed frequency and intensity, while the radial temperature law stays the same. Tests verify reversal swaps Doppler shifts without changing local T .

Q20. What is physically validated?

Vacuum geodesics, null initialization, invariants, orbital plane, photon sphere, critical b , capture/escape, weak deflection, shadow angle, numerical convergence, and CPU/GPU comparisons within documented tolerances.

Q21. What is not physically complete?

Disk plasma, MHD, absorption, scattering, spectral transfer, polarization, Kerr spin, and detector response.

Q22. What does Cinematic mode change?

Only presentation/emission choices: orange grading, artificial photon-ring glow, higher-order brightness boost, and filmic tone mapping. Ray geometry is unchanged.

Q23. What is Flamm's paraboloid?

An embedding diagram of one constant-time equatorial spatial slice. It is not the path rays move on and not the full spacetime.

Q24. Why keep unresolved rays separate from captured rays?

Otherwise numerical failures would create false black pixels and bias the measured shadow.

Q25. Why have a CPU reference if rendering is on the GPU?

The readable double-precision implementation enables analytic tests and an independent comparison against production float code.

Q26. What would be the most important next step?

For visual astrophysics, Kerr geodesics plus frequency-dependent optically thin radiative transfer would be a major increase in realism.

16. Short technical report

Title

Validated GPU Visualization of Null Geodesics in Schwarzschild Spacetime with an Approximate Thin Accretion-Disk Emitter

Abstract

This project implements a real-time C++17/OpenGL visualization of light propagation around a nonrotating, uncharged black hole. A compute shader traces one backward null ray per image pixel in dimensionless Schwarzschild coordinates. Rays are initialized from a static-observer orthonormal tetrad and integrated with fourth-order Runge-Kutta plus step-doubling error control. Outcomes are classified as captured, escaped, disk-hit, unresolved, or invalid. Captured rays form the shadow; escaped rays sample a lensed procedural sky; disk hits use a simplified zero-torque thin-disk effective temperature, four-vector frequency shift, and bolometric g^4 intensity scaling. A dependency-free double-precision CPU implementation provides the reference for analytic and GPU comparisons.

Objectives

- Render qualitatively and quantitatively correct Schwarzschild vacuum lensing in real time.
- Keep physical and artistic presentation modes explicitly separated.
- Validate initial conditions, conserved quantities, critical geodesics, shadow geometry, integration order, and disk thermodynamic calculations.
- Expose numerical failures rather than merging them into the black-hole shadow.

Method

Camera rays are constructed in a local static tetrad and converted to Schwarzschild coordinate derivatives. The production shader evolves (r, θ, ϕ) and their affine derivatives, reconstructing t' from conserved E . Its radial acceleration uses the null-constraint-reduced expression $L^2(r-1.5)/r^4$. RK4 step doubling compares one full step with two half-steps using absolute and relative tolerances. Geometric caps protect the horizon and strong-curvature region. Segment interpolation detects horizon and finite-disk crossings. The GPU executes 16×16 work groups and writes an RGBA8 texture displayed on a fullscreen quad.

Emission model

The disk extends from 3 to 12 r_s with configurable inclination and half-thickness. Its local temperature is derived from $F = 3GMM\dot{[1-\sqrt{r_{in}/r}]]/(8\pi r^3)$ and the Stefan-Boltzmann law. Observed frequency is obtained from $g = (-k_{u_o})/(-k_{u_e})$ for a static observer and circular emitter; bolometric intensity is multiplied by g^4 . Procedural detail changes emissivity only. Cinematic mode adds nonphysical ring emission and grading.

Results

The locally rerun CPU suite passed all checks. Maximum initial null residual was 6.66×10^{-16} ; energy drift 3.77×10^{-15} ; L^2 drift 5.61×10^{-13} . The photon sphere remained at 1.5 r_s , b_{crit} equaled 2.59807621135 r_s , weak-field deflection at $b=100 r_s$ was 0.020299965 rad, the default shadow half-angle was 27.7071984 degrees, and RK4 step halving reduced error by 15.9824. The disk temperature at 4.5 r_s was 5866.05094 K and tenfold accretion increased temperature by 1.77827941. The supplied audit additionally records passing GPU comparisons and zero unresolved/invalid rays in its deterministic runs.

Conclusion

The system is best described as a validated Schwarzschild null-geodesic renderer with an approximate finite-thickness disk emitter. It is suitable for demonstrating gravitational lensing, shadows, photon-sphere sensitivity, relativistic shifts, GPU numerical integration, and scientific software validation. It should not be presented as a full prediction of a specific astrophysical black hole because spin, plasma dynamics, and radiative transfer are omitted.

17. Glossary

Term	Definition
Affine parameter	Parameter used to move along a geodesic without changing the geometric path.
Bolometric	Integrated over all frequencies rather than a specific detector band.
Capture surface	Numerical stopping radius just outside the horizon.
Christoffel symbols	Coordinate-dependent coefficients that appear in the geodesic equation.
Compute shader	GPU program for general parallel computation rather than traditional vertex/fragment rasterization.
Event horizon	Boundary beyond which future light cannot escape.
Four-vector	Object with time and three spatial components that transforms consistently in relativity.
Frame dragging	Rotation-induced twisting of spacetime; absent in Schwarzschild and present in Kerr.
Geodesic	Straightest possible path in curved spacetime.
GLFW	Library for windows, input, and OpenGL context creation.
GLEW	Library that loads OpenGL functions.
GLM	C++ mathematics library with vector and matrix types.
ImGui	Immediate-mode graphical user-interface library used for the HUD.
Impact parameter	Angular momentum divided by energy for a photon; roughly its miss distance far away.
Invariant	Quantity independent of coordinates or conserved by a symmetry; used as a diagnostic.
ISCO	Innermost stable circular orbit for massive matter.
Kerr black hole	Rotating black-hole solution; not implemented here.
Metric	Rule that measures spacetime intervals and defines causal structure.
Null	Lightlike: spacetime interval equals zero.
Photon sphere	Unstable circular-light radius in Schwarzschild spacetime.
Radiative transfer	Transport of radiation including emission, absorption, and scattering.
Ray tracing	Following idealized light paths to determine image formation.
RK4	Fourth-order Runge-Kutta numerical integrator.
Schwarzschild radius	$2GM/c^2$; the horizon radius for the modeled black hole.
SSBO	Shader storage buffer object; GPU-readable/writable array storage used for diagnostics.
Tetrad	Local orthonormal basis used to connect a physical observer's measurements with coordinates.
UBO	Uniform buffer object; read-only configuration data shared by shader invocations.

18. Formula and source traceability

These references point to the supplied archive after extraction. Line numbers are included so you can open the source and connect each explanation to implementation.

Topic	Primary source location
Configuration defaults and validity	physics/simulation_config.hpp:15-75; physics/simulation_config.cpp:10-45
Schwarzschild radius and metric factor	physics/schwarzschild.cpp:96-109
Static-observer tetrad	physics/schwarzschild.cpp:111-142; geodesic.comp:459-505
Null norm and invariants	physics/schwarzschild.cpp:144-173; geodesic.comp:574-583
CPU geodesic RHS	physics/schwarzschild.cpp:176-197
GPU reduced radial equation	geodesic.comp:511-525
RK4	physics/schwarzschild.cpp:200-208; geodesic.comp:538-565
Adaptive step doubling	physics/schwarzschild.cpp:211-296; geodesic.comp:586-641
Disk intersection	geodesic.comp:223-258
Disk temperature and frequency shift	physics/accretion_disk.cpp:13-56; geodesic.comp:277-313
Disk shading and g^4	geodesic.comp:314-428
Ray classification	geodesic.comp:678-853
Compute dispatch and UBO upload	black_hole.cpp:311-517
Flamm's paraboloid	black_hole.cpp:192-269
Camera controls	app/camera.cpp:22-128
CPU validation cases	tests/schwarzschild_validation.cpp
GPU comparison logic	tests/gpu_validation_compare.cpp
Documented scope	README.md; PHYSICS_VALIDATION.md; VALIDATION_REPORT.md

18.1 Internal references used

- README.md - user scope, build/run instructions, controls, architecture, and troubleshooting.
- PHYSICS_VALIDATION.md - metric, conventions, equations, validation tolerances, evidence, and limitations.
- VALIDATION_REPORT.md - dated clean-build and hardware audit supplied with the project.
- Source files under physics/, app/, rendering/, tests/, plus black_hole.cpp and geodesic.comp.

Final presentation advice

Lead with the claim that is strongest and easiest to defend: **validated Schwarzschild null-geodesic renderer with an approximate thin-disk emitter**. Then show that you understand the boundary between geometry, numerics, GPU implementation, and artistic presentation.

End of guide